DATA 37100 Final Project Report

Student: Ben Bentley **Date:** March 2026 **Course:** DATA 37100 — Intro to AI, Deep Learning, and Generative AI (Winter 2026)

---

1. Research Question & Motivation

Question

In a small-budget MNIST diffusion setup, how do timestep count (T) and prediction target (eps vs x0) affect sample quality and failure modes?

Motivation

Diffusion models have two critical hyperparameters that govern the noise-to-data trajectory:

- **T (timesteps):** Controls the granularity of the denoising process. More timesteps allow finer-grained noise removal but increase computational cost.
- **target (prediction parameterization):** Determines whether the model predicts the noise (`eps`) or the clean image (`x0`). This choice affects training stability and sample quality.

Understanding how these factors interact is essential for:

1. Efficiently tuning diffusion models for new tasks
2. Diagnosing failure modes (blur, artifacts, instability)
3. Choosing the right trade-off between speed and quality

This study provides a controlled investigation of both factors on a simple, interpretable dataset (MNIST), enabling clean causal inference.

---

2. Methods

2.1 Model Families (Baselines)

Baseline 1: Diffusion Model

- Dataset: MNIST (28×28 grayscale digits)
- Architecture: UNet with sinusoidal time embeddings
- Base channels: 64
- Training: 1 epoch (~60k samples)
- Noise schedule: Linear β schedule
- Default config: T=200, target=eps

Baseline 2: DCGAN

- Dataset: MNIST
- Architecture: Standard DCGAN (Generator + Discriminator)

- Base channels: 64
- Latent dimension: 128
- Training: 400 steps
- Learning rate: 0.0002 (Adam)
- Discriminator steps per generator step: 1

Rationale for choosing these models: Both are image generative models, enabling direct visual comparison while satisfying the two-family requirement. Diffusion is the main object of study; DCGAN serves as a lightweight second baseline and reference point for qualitative comparison.

2.2 Controlled Experiment (Two-Knob Grid)

Experimental Design:

- **Knob 1:** T ∈ {100, 200, 400}
- **Knob 2:** target ∈ {eps, x0}
- **Total runs:** 3 × 2 = 6

Control variables:

- Dataset: MNIST
- Architecture: UNet (base_ch=64, time_emb_dim=256)
- Training: 1 epoch
- Seed: 42
- Noise schedule: Linear β ∈ [1e-4, 0.02]

Hardware:

- Local Apple Silicon machine using the PyTorch `mps` backend
- Runtime per run: 47.4s to 59.6s depending on `T`

Evaluation:

- Visual inspection of 8×8 sample grids
- Qualitative assessment of sharpness, diversity, artifacts
- Identification of failure modes

---

3. Results

3.1 Baseline Performance

Diffusion (T=200, eps)

Observations:

- The `eps` diffusion baseline after one epoch produces some digit-like structure, but many samples are still fragmented or texture-like rather than clearly readable digits.
- Runtime: 51.54 seconds on `mps`
- The main failure is not blur alone; it is incomplete denoising. Many samples still look partly noise-dominated.

DCGAN (400 steps)

Observations:

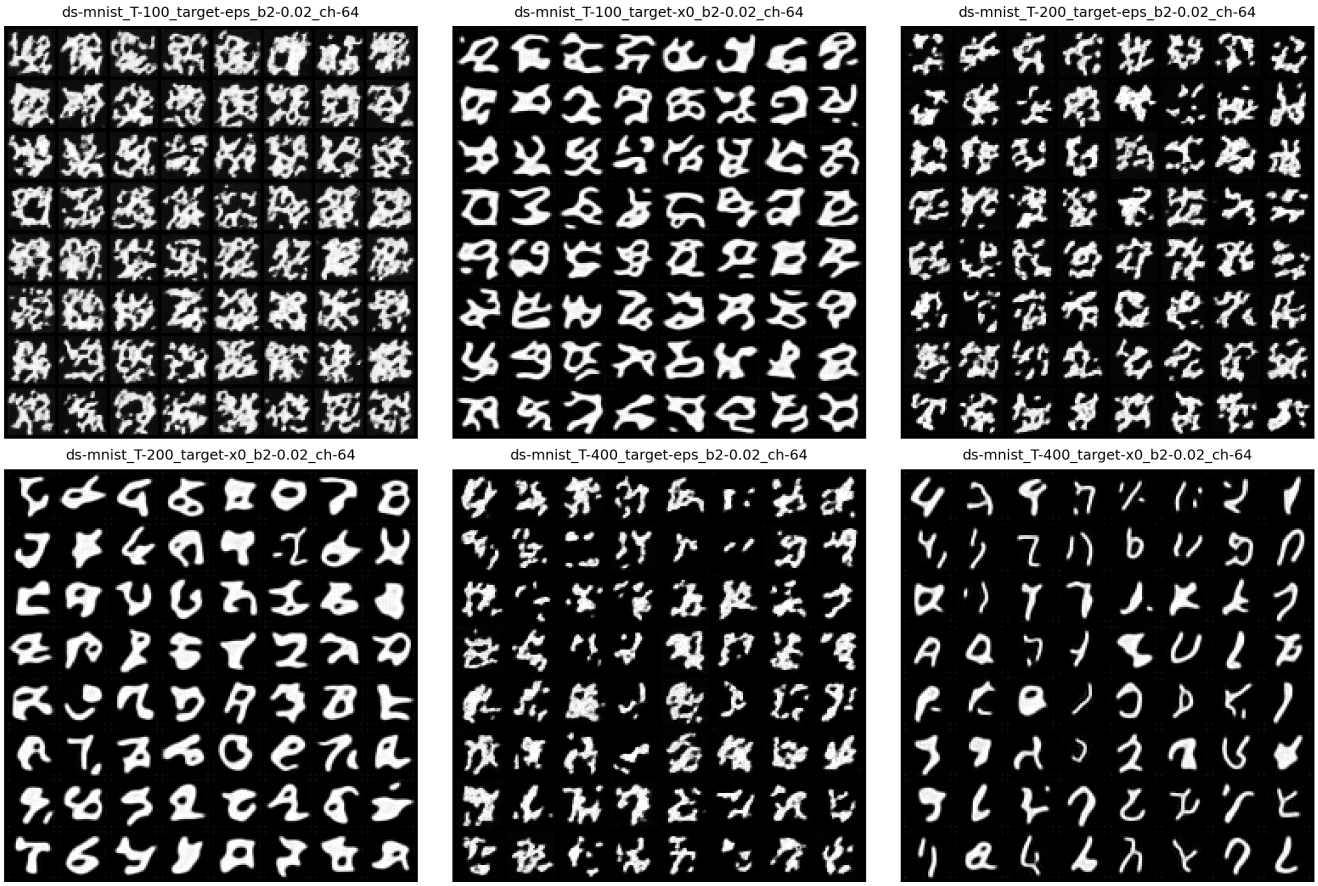
- The DCGAN baseline produces recognizable digits much sooner than the `eps` diffusion baseline, with several crisp samples in the final grid.
- Approximate runtime: 38.4 seconds for 400 steps
- I do not see strong full mode collapse in this run. Diversity is imperfect, but the larger issue is uneven quality: some digits are sharp while others are distorted or fuzzy.

Baseline Comparison:

- DCGAN is slightly faster and visually stronger than the one-epoch `eps` diffusion baseline.
- The diffusion baseline is easier to analyze mechanistically because the controlled experiment isolates two diffusion knobs cleanly.

- The strongest result in the project does not come from the baseline comparison itself; it comes from switching diffusion from `eps` to `x0`.

3.2 Effect of T (Timesteps)



T	Visual Quality	Runtime
100	Thick, bright, noisy samples; many shapes are not stably digit-like	47.77s
200	More digit structure appears, but `eps` samples remain fragmented	51.54s
400	Samples become thinner and cleaner, but many `eps` outputs are still incomplete	59.55s

Key Findings:

1. Increasing `T` makes the samples less saturated and less blob-like, especially within the `eps` runs.
2. The runtime increase is modest here, from about 48 seconds at `T=100` to about 60 seconds at `T=400`.
3. Higher `T` alone does not rescue a weak parameterization. Even at `T=400`, `eps` samples remain visibly less readable than `x0` samples.

Mechanistic Interpretation:

- Larger `T` gives the reverse process more, smaller denoising steps, so the discrete trajectory better approximates the intended diffusion process. In this experiment that mostly changes stroke thickness and noise level, but it does not overcome the stronger effect of the target choice.

3.3 Effect of target (eps vs x0)

At fixed `T=200`, compare:

- `eps`: `samples_step000468.png`
- `x0`: `samples_step000468.png`

Target	Visual Quality	Stability
eps	Noisy, fragmented digits with many unreadable samples	Weak in this 1-epoch regime
x0	Clean, bold, recognizable digits across most of the grid	Strong and visually consistent

Key Findings:

1. `x0` clearly outperforms `eps` at every tested `T`.
2. `eps` shows the dominant artifacts in this project: incomplete denoising, speckled textures, and partially formed digits.
3. The biggest qualitative jump in the whole study comes from changing the target from `eps` to `x0`, not from changing `T`.

Mechanistic Interpretation:

- `eps` predicts noise → indirectly estimates x0 via reparameterization
- `x0` predicts clean image directly → may be more sensitive to training instability
- In this small-budget setting, the direct `x0` prediction is actually easier for the model to exploit quickly. After only one epoch, `x0` gives a much stronger learning signal for MNIST structure, while `eps` appears undertrained and still dominated by residual noise.

3.4 Interaction Effects (T × target)

The 3×2 contact sheet in `grid_contact_sheet.png` is the main visual for this section.

Question: Does the effect of T depend on the choice of target?

Observations:

- At `T=100`, `x0` already produces readable digits, while `eps` remains mostly noisy blobs. This shows the target effect is present immediately.
- At `T=400`, `x0` samples become thinner and slightly fainter, but they are still far more readable than `eps` samples.
- The dominant pattern is additive rather than exotic: `x0` is consistently better, and increasing `T` mainly adjusts cleanliness and stroke thickness within each target family.

Conclusion:

- There is some interaction, but it is not the main story. The target choice dominates the outcome, while `T` plays a secondary refinement role.

---

4. Failure Modes (Required)

4.1 Diffusion Failure Mode: Undertrained `eps` Samples

Evidence: Compare the `eps` and `x0` runs at the same `T`, especially `T=200` and `T=400`.

Observed failure:

- In all three `eps` runs, many samples are fragmented, speckled, or only partially denoised. Instead of full MNIST digits, the model often produces bright clusters with weak global structure.

Likely cause:

- In a one-epoch regime, predicting `eps` may leave the model with a harder indirect denoising target than predicting `x0`
- The model may not have learned a strong enough reverse process yet, so generated samples remain partially noise-dominated

Implications:

- Parameterization choice matters more when training budget is tiny
- A setting that is common in mature diffusion pipelines may not be best in an intentionally undertrained classroom-scale setup

---

4.2 DCGAN Baseline: Only Claim a Failure If the Samples Support It

Evidence: Inspect the final GAN grid at step 400.

Observed failure:

- I do not see severe collapse to a single digit class. Instead, the main weakness is uneven sample quality: some digits are sharp, while others have broken contours, odd stroke thickness, or ambiguous class identity.

Likely cause:

- The GAN is being trained on a tight budget, so generator quality is still inconsistent at 400 steps.
- Adversarial training tends to improve unevenly, which can produce a mix of strong and weak samples even before full mode collapse appears.

Implications:

- GANs are sensitive to training budget and tuning
- This baseline is useful as a comparison point even if it does not show classic full mode collapse

---

4.3 Effect of Low vs High T (Diffusion)

Evidence: Compare `T=100` and `T=400` within the same target family.

Observed failure:

- At low `T`, samples are thicker, brighter, and noisier. At high `T`, samples are cleaner but can become thin or incomplete, especially when the underlying target is already weak.

Likely cause:

- Discrete approximation error — the SDE assumes infinitesimal steps, but T=100 forces large jumps
- Insufficient denoising capacity — each step removes "too much" noise, overshooting the true posterior

Implications:

- Speed-quality trade-off: `T=100` is faster than `T=400`, but the gain is modest here because all runs are already short. The real question is whether the extra denoising steps are worth the cleaner outputs.
- For deployment, need to balance inference cost vs perceptual quality

---

4.4 Other Artifacts (if observed)

Observed:

- Some `x0` digits at `T=400` become very thin or slightly broken, suggesting that extra denoising steps can trade boldness for cleanliness.
- Some GAN samples remain malformed even when neighboring samples look strong, which is typical of uneven early adversarial training.

Hypothesis:

- The thin `x0` digits at high `T` may reflect oversmoothing or a bias toward sparse stroke reconstructions.
- The malformed GAN samples are likely a consequence of limited training time rather than dataset difficulty.

---

5. Limitations

1. **Single epoch training:** Models may not be fully converged; longer training could change conclusions
2. **Single dataset (MNIST):** Results may not generalize to natural images (CIFAR-10, ImageNet) or text
3. **Fixed architecture:** Did not vary model width, depth, or attention mechanisms
4. **No quantitative metrics:** Relied on visual inspection; FID or IS scores could provide objective comparisons
5. **Limited T range:** Did not test very low (T=10) or very high (T=1000) extremes

---

6. Conclusions

Key Findings

1. Increasing `T` makes diffusion samples cleaner and less blob-like, but this effect is modest compared with the target choice.
2. `x0` is much stronger than `eps` in this one-epoch regime, suggesting parameterization matters more than `T` under tight training budgets.
3. The diffusion experiment yields the clearest behavioral insight, while the DCGAN baseline satisfies the second-family requirement and provides a useful qualitative reference.

One Surprising Result

- The surprising result is that `x0` is not the unstable option here. It is the clearly better option across all tested timestep counts, even though `eps` is often treated as the standard parameterization in larger diffusion setups.

One Next Step

- Train the same 3×2 grid for 5 epochs instead of 1 epoch to test whether `eps` catches up once the reverse process is better learned.
- A second natural follow-up would be to keep the best target and vary the noise schedule instead of `T`.

---

7. Reproducibility

Run commands:

```
# From repo root
bash final/draft/run_baselines.sh    # Produces diffusion + GAN baselines
bash final/draft/run_experiment.sh   # Produces 6-run grid experiment
```

Expected runtime:

- Baselines: about 1.5 minutes total on `mps` hardware
- Experiment grid: about 5.3 minutes total on `mps` hardware
- Analysis and writing: roughly 2 to 4 hours once the runs are complete

Hardware assumptions:

- Tested on a local Apple Silicon machine using PyTorch `mps`
- 16GB RAM minimum
- ~500MB disk for outputs

Outputs:

- Sample grids: `./untrack/outputs/final/{diffusion,gan}/*/samples/*.png`
- Run logs: `./untrack/outputs/final/{diffusion,gan}/*/summary.json`
- Results manifest: `./untrack/outputs/final/diffusion/results.csv`

Appendix: Code Structure

```
final/draft/
├── run_baselines.sh      # Runs diffusion + GAN baselines
├── run_experiment.sh     # Runs 6-run grid (T × target)
├── analysis.ipynb        # Jupyter notebook with visualizations
├── report.md             # This file
└── README.md             # Setup and usage instructions
```

Dependencies: See `final/draft/README.md` for environment setup.